CENGAMES / PLAN DE PRODUCTO Y MOTOR

Juego de peleas por equipos

Combate local de equipos 3 contra 3 frente a una CPU que aprende, con un motor independiente de Flappy BirdCen.

ALCANCE

Una persona contra CPU. Sin juego en línea.

EQUIPO

Tres peleadores por lado. Diez personajes seleccionables.

MOTOR

Canvas y JavaScript nativo, sin acoplar reglas a sprites.

RECOMPENSA

5 Cent por combate completo + 2 Cent por victoria.

DIRECCIÓN VISUAL

Pixel art neo-retro inspirado en arcades de 16 bits con interfaz moderna tipo iOS.

Versión 1.0 - base de diseño

Referencia para construir el motor antes de integrar los sprites finales.

1. Resumen y decisiones cerradas

Este documento fija el alcance inicial y anticipa los riesgos que conviene resolver antes de producir diez personajes. Primero se prueba combate, controles y CPU con figuras provisionales; después se conectan los sprites sin rehacer la simulación.

> **Juego local de una persona contra la computadora. Equipos de tres, eliminación uno contra uno, hasta diez personajes seleccionables. Sin matchmaking, cuentas en línea ni servicio remoto.**

Decisiones aprobadas

Tema	Regla de la primera versión
Modo	Una persona contra CPU.
Equipo	Escoger 3 de 10 personajes, sin repetir y con orden de entrada.
Campo	Un peleador activo por equipo; reservas fuera de pantalla.
Victoria	Gana quien elimine a los tres rivales. Empate si ambos equipos pierden al último peleador en el mismo evento.
Duración	40 s por duelo; 200 s de relojes base + hasta 25 s de muerte súbita y transiciones.
CPU	Muy fácil, normal y difícil; mismas reglas, vida, daño y movimientos.
Cent	5 por terminar + 2 por victoria. Empate: 5. Abandono: 0.
Arte	Sprites se adaptan después de cerrar el contrato de animación.

Alcance inicial

Multijugador, ranking en la nube, compras de personajes, desbloqueos, combos largos y asistencias quedan fuera de la primera versión. La arquitectura deja puntos de extensión para esas ideas sin programarlas ahora.

2. Lectura del proyecto actual

CenGames es una aplicación estática de HTML, CSS y JavaScript sin dependencias. El README dice que el sitio publicado guarda perfiles, Cent y estadísticas en el navegador; SQLite es una API opcional de desarrollo local. El juego de peleas debe funcionar con ese mismo modelo.

Piezas reutilizables

- Lobby, perfil, saldo compartido de Cent, tema y dialogs.
- Canvas 2D, requestAnimationFrame, escala por densidad de pantalla y pausa al ocultar la página.
- Archivos estáticos y desarrollo sin proceso de compilación.

Piezas que son de Flappy

- Flappy tiene una sola pantalla vertical y controles de aleteo.
- Renderizadores, audio, score y eventos actuales están nombrados y ligados a Flappy.
- CenEconomy paga Cent por puntos de Flappy; la API solo acepta gameId flappy y guarda sus estadísticas.
- El service worker enumera scripts y arte para cache offline. Hay que agregar recursos nuevos y cambiar la versión de cache.

Límite operativo: en el sitio publicado los datos son locales, no hay cuenta segura ni sincronización entre dispositivos. El saldo local se puede editar desde el navegador; el README ya describe esta limitación.

Frontera nueva

El juego vivirá en src/js/fighter/ con pantalla y estilos propios. El router del portal solo inicia y cierra el juego. CenEconomy será la única pieza compartida para perfil y recompensa; la API SQLite local se amplía solo para desarrollo.

3. Reglas del combate por equipos

Elección de escuadras

- Cuadrícula de diez personajes, todos seleccionables al inicio.
- Elegir tres distintos y ordenar la entrada. No se repiten dentro del mismo equipo; se permiten espejos entre equipos.
- La CPU elige tres y su orden; guardar una semilla de partida para poder repetir y depurar.
- La elección se guarda por perfil y por juego, separada del personaje de Flappy.

Flujo de partida

Paso	Evento y resultado
Selección	Jugador elige equipo/orden; CPU arma su escuadra.
Duelo	Un personaje por lado empieza con 100% de vida; cada duelo tiene reloj nuevo de 40 s.
Nocaut	El derrotado queda eliminado; entra el siguiente con vida completa.
Superviviente	Conserva vida y enfriamiento del especial; entra neutralizado (sin stun ni velocidad), sin curación.
Transición	Hasta 1 s; se congela el combate. El reloj y los enfriamientos corren solo durante el duelo.
Tiempo	A 0 segundos, mayor porcentaje de vida sobrevive y el rival sale.
Empate de vida	Muerte súbita de hasta 5 s; el primer ataque que toca al rival decide, aunque lo bloquee; sin contacto ambos salen.
Fin	El equipo que pierde sus tres personajes pierde. Doble KO final es empate.
Premio	Una sola entrega de Cent por ID de la partida completa.

Duración objetivo

Un duelo dura hasta 40 segundos. La partida puede terminar en tres duelos si un superviviente barre a la escuadra rival; una pelea cerrada puede llegar a cinco. Límite: hasta 5 duelos de 40 s y hasta 5 s de muerte súbita por duelo; 225 s de combate máximo + hasta 4 transiciones de 1 s (3:49), más intro y resultado. Meta de experiencia: 2 a 3 minutos.

> **Un personaje derrotado no regresa. El superviviente avanza con la vida restante. Esto recompensa conservar vida y evita curaciones gratuitas entre rivales.**

4. Combate y controles

Reglas que pertenecen al motor

- Arena lateral 2D con suelo, gravedad, límites y salto.
- Los peleadores miran al rival; al cruzarse voltean sin teletransportarse.
- Cada movimiento declara preparación, ventana activa y recuperación. Un ataque registra como máximo un impacto por objetivo.
- Defensa reduce daño y avance; un daño chip pequeño no letal evita bloqueo infinito.
- Stun y retroceso interrumpen ataques; no se puede golpear durante stun o recuperación.
- Tres acciones ofensivas de inicio: ligero, fuerte y especial por personaje. Combos largos y cancelaciones llegan después del balance básico.

Valores iniciales ajustables

Sistema	Base para prototipo	Perilla de balance
Vida	100 puntos por personaje	Vida/arquetipo
Reloj	40 s por duelo	Duración y ritmo
Ligero	Rápido, daño bajo, recuperación corta	Startup/alcance
Fuerte	Más lento, daño y retroceso mayores	Castigo y knockback
Especial	Habilidad propia con enfriamiento	Efecto y enfriamiento
Bloqueo	Reduce daño; chip no letal	Reducción y stun
Simulación	60 ticks por segundo	Cadencia fija

Controles

Teclado: A/D mover, W saltar, S agacharse, Shift bloquear, J ligero, K fuerte, L especial, Escape o P pausar. **Teléfono:** pad direccional a la izquierda; salto, bloqueo y tres ataques a la derecha. El diseño admite varias teclas o dedos, limpia entradas al perder foco y evita el scroll sobre el campo.

Un buffer corto conserva un ataque pulsado durante una recuperación para mejorar sensibilidad. El control del jugador no recibe demora artificial por la dificultad de CPU.

5. CPU con aprendizaje local

El motor decide qué acciones son legales; el controlador escoge qué intentar. Una tabla Q-learning compacta parte de una política entrenada en simulaciones y aprende preferencias por perfil tras partidas completas. Usa señales acotadas por acierto, bloqueo, castigo recibido y resultado. La dificultad limita reacción y calidad dentro de una envolvente fija: aprender no permite que Muy fácil supere la respuesta de Difícil. No controla colisiones, no cambia daño y no conoce entradas futuras.

Enfoque de machine learning

- Política inicial entrenada fuera de partida con simulaciones contra distintos estilos.
- Modelo compacto de aprendizaje por refuerzo con estados discretos: distancia, borde, estado propio, fase del rival, vida relativa, enfriamiento y tendencias recientes.
- Acciones: acercarse, retroceder, cubrirse, saltar, ligero, fuerte o especial. El motor descarta acciones imposibles.
- Registra aciertos, bloqueos y fallos; ajusta preferencias al cerrar la partida completa. Limita cada cambio para evitar que el modelo se rompa o que reiniciar partidas abuse del aprendizaje.
- Aprendizaje local por perfil, versionado y reiniciable; no requiere internet ni envía telemetría.

Dificultades

Modo	Latencia inicial a 60 Hz	Conducta
Muy fácil	24 cuadros, aprox. 400 ms	Deja pasar oportunidades; se equivoca con frecuencia.
Normal	12 cuadros, aprox. 200 ms	Gestiona distancia, defensa y castigos evidentes.
Difícil	6 cuadros, aprox. 100 ms	Aprovecha recuperaciones y varía respuesta a hábitos.

La diferencia de modo viene de reacción y calidad de decisión, no de vida o daño. La política no responde antes de que una acción sea observable. Los tres modos se prueban con los mismos personajes, semillas y escenarios; medir tasa de victoria, duración y daño recibido.

Riesgo de ML: demasiadas combinaciones de estado producen aprendizaje disperso. Empezar con pocas categorías, una política base compartida por arquetipos y ajustes locales pequeños.

TensorFlow.js permite ejecutar y entrenar modelos en navegador mediante una biblioteca. Para respetar el proyecto estático y sin dependencias, se prefiere una política compacta escrita en JavaScript puro. Lecturas oficiales: tutoriales de TensorFlow.js y opciones de configuración.

6. Arquitectura independiente

La simulación debe correr sin DOM, Canvas ni sprites. Así podemos entrenar la CPU por lotes en Node, probar reglas deterministas y usar las mismas reglas en el navegador.

Módulo	Responsabilidad
fighter-config.js	Datos de diez personajes, movimientos, límites y dificultad.
fighter-engine.js	Reloj, estado, física, colisiones, vida, eliminaciones y resultado. Sin dibujo.
fighter-input.js	Teclado/táctil a acciones normalizadas; limpia estados al pausar.
fighter-ai.js	Observa el estado, decide acciones legales, aplica ML y dificultad.
fighter-renderer.js	Dibuja arena, animación, HUD y efectos; nunca cambia las reglas.
fighter-game.js	Ciclo de vida RAF, pausa, foco, resize, reinicio y salida.
HTML y fighter.css	Selección de equipo, menús, HUD, resultado y controles táctiles.

Contratos de datos

FighterDefinition: id, nombre, arquetipo, atributos, movimientos y atlas. **FighterState:** posición, velocidad, orientación, vida, enfriamiento, estado y movimiento activo. **MoveDefinition:** startup/active/recovery en cuadros, hitbox, daño, stun y retroceso. **MatchState:** equipos, orden, activos, reloj, resultado e ID.

Ciclo de vida

Entrada	Simulación	Salida
Jugador + CPU	Avance fijo a 60 Hz	Estado para renderer y HUD

Los dialogs, pestaña oculta y cambio de pantalla pausan. Al salir se cancela RAF, se quitan listeners y se borran teclas mantenidas. Reanudar exige una acción explícita.

> **No crear un motor genérico para todos los minijuegos. Crear una frontera pequeña entre portal, motor, CPU, arte y economía; conservar intactos los módulos de Flappy.**

7. Dirección visual y sprites

Opinión estética

Recomiendo **neo-retro de 16 bits** para la arena y los personajes, con una interfaz moderna inspirada en iOS. El arte pixelado puede usar poses expresivas, y el HUD puede usar paneles de vidrio oscuro, bordes suaves, tipografía limpia y acentos de color de CenGames.

- 8-bit estricto limita siluetas, expresiones y cuadros de movimiento para diez luchadores. El estilo de 16 bits conserva nostalgia y da lectura suficiente a los ataques.
- La referencia es el género de peleas arcade: nombres, personajes, movimientos, sprites y audio serán originales o se usarán con permiso.
- Escalado nearest-neighbor para sprites nítidos; nada de blur en el personaje.
- Fondos con parallax y efectos breves; prioridad a leer el golpe, el retroceso y el estado.
- Paleta base teal, crema, dorado y coral; cada peleador tiene color secundario. Vida/estado se leen también por forma, icono y texto.
- HUD tipo vidrio con alto contraste, botones táctiles grandes, zonas seguras y opción de movimiento reducido.

Contrato para recibir y adaptar sprites

Elemento	Regla
Atlas	PNG transparente por personaje, celdas consistentes y manifiesto de filas/cuadros.
Ancla	Pies alineados para que no salte el cuerpo al cambiar animación.
Animaciones	Idle, caminar, salto, agacharse, defensa, ligero, fuerte, especial, golpe, caída, levantar y KO.
Orientación	Un lado y espejo horizontal; declarar excepciones asimétricas.
Hitboxes	Datos por movimiento/cuadro, independientes del arte.
Producción	Completar una pareja como prueba visual; luego escalar en lotes hasta diez.

La arena usa una relación adaptable cercana a 16:9. En teléfono se recomienda jugar en horizontal; en vertical se mantiene una vista y controles utilizables y solo se sugiere girar el dispositivo. No forzar orientación. La resolución interna y las celdas finales se deciden al inspeccionar los sprites fuente. Así se evita recortar cuerpos o forzar una rejilla incompatible.

8. Cent, estadísticas y modo local

Recompensa aprobada

Resultado de partida completa	Cent
Victoria	7 total: 5 por completar + 2 bono
Derrota	5
Empate	5
Abandono antes del resultado	0

La unidad pagada es la partida completa de equipo, no cada golpe, KO o duelo. matchId es único; se guarda con el recibo y las estadísticas en la misma actualización del perfil local. Repetir el guardado no duplica saldo.

Perfil del navegador

- Guardar victorias, derrotas, empates, racha y Cent ganados bajo el namespace del nuevo juego.
- Guardar por ID local del perfil; separar roster, opciones y aprendizaje.
- Persistir eventos resumidos y parámetros del modelo, no una grabación completa de teclas.
- La web publicada sigue jugable con API apagada; no hay sincronización entre dispositivos.

SQLite opcional de desarrollo

Agregar operación de resultado y estadísticas gameId fighter con migración aditiva. La API local calcula el premio fijo y deduplica matchId en una transacción. El navegador guarda recibo, saldo y estadísticas en una sola escritura del perfil; se puede jugar igual con la API apagada. Si no se puede guardar, mostrar el error y ofrecer reintento usando el mismo matchId. Si la respuesta de la API local es incierta, reintentar con el mismo matchId y no crear una concesión local distinta.

> **No reutilizar la ruta actual de Flappy, que suma puntos como Cent y acepta gameId flappy. Crear una acción distinta y actualizar README.md. Los Cent son ficticios; el perfil local no está protegido contra edición.**

Privacidad y almacenamiento

La política ML guarda preferencias resumidas por jugador local. Debe tener versión, límite de tamaño y opción de reinicio. Si el navegador rechaza el almacenamiento, la partida sigue con política base, pero el aprendizaje no persiste. No enviar datos fuera del navegador; el proyecto no requiere red para combatir.

9. Riesgos y prevención

Riesgo	Mitigación
ML repite una acción dominante	Entrenamiento diverso, premios por eventos, límites de ajuste y política base.
Dificultad injusta	Mismas reglas y atributos; medir reacción; no anticipar acciones privadas.
Partida se alarga	40 s por duelo, vida decide timeout y muerte súbita corta.
Personaje dominante	Datos de arquetipo configurables; pruebas por matchup y estadística por personaje.
Hitbox no coincide con sprite	Overlay de debug; frame data separado; revisar atlas y ancla con cada movimiento.
Táctil pegado o scroll	Captura/cancel de puntero; reset al blur, pausa y cambio de pantalla.
Bajo rendimiento	DPR limitado, catch-up limitado, efectos escalables y simulación fija.
Cache mezclada	Lista explícita del service worker, versión de cache y prueba offline.
Pago duplicado	Un solo estado terminal, ID idempotente y recompensa derivada del resultado.

Reglas cerradas para casos límite

- Ataques que conectan en el mismo tick se resuelven a la vez; ambos daños aplican. Si ambos caen, ambos quedan eliminados y el combate continúa o termina en empate si fue el KO final.
- La transición dura hasta 1 s sin ataques y congela física, reloj y enfriamiento. El sustituto entra con vida completa; el superviviente conserva vida y enfriamiento, pero vuelve a estado neutral.
- Paredes limitan posición y retroceso; al tocarse los luchadores pueden cruzarse y se voltean, sin teletransportes.
- Timeout empatado: muerte súbita de 5 s; el primer ataque que toca al rival decide, aunque lo bloquee. Sin contacto se elimina a ambos; un intercambio simultáneo también elimina a ambos.
- Escape abre pausa y confirmación de abandono. Recargar o cerrar durante combate cuenta como abandono: 0 Cent y sin reanudación. El resultado y recibo se guardan antes de mostrar la pantalla final; matchId deduplica el premio.
- La semilla de CPU queda en el registro local para reproducir incidencias; no se guarda video ni la secuencia completa de teclas.

Futuro sin ampliar alcance

Gamepad, práctica, repetición local, torneos locales, cosméticos y relevos voluntarios pueden incorporarse más adelante. Multijugador en línea implica otra infraestructura y no forma parte de esta Versión local.

10. Fases y criterios de salida

Fase	Entregable
1. Motor puro	Simulación sin DOM ni arte: movimiento, hit, daño, colisión y reloj.
2. Equipos	Selección 3, cola de entrada, vida persistente del superviviente, timeout y resultado.
3. CPU	Acciones legales y tres dificultades configurables.
4. ML	Entrenamiento reproducible, modelo base versionado y aprendizaje local reiniciable.
5. Integración	Pantalla de lobby, HUD, pausa, controles táctiles y Cent idempotentes.
6. Pareja de arte	Probar sprites, manifiesto, anclas, clips y hitbox debug.
7. Diez peleadores	Adaptar sprites por lotes, balancear matchups y terminar audio/efectos.
8. Publicación	Service worker, README, tests y QA desktop/móvil/offline.

Definition of done del motor antes de sprites finales

- Dos figuras de prueba pueden moverse, bloquear y conectar ataques de forma repetible.
- Cada ataque respeta startup/active/recovery y no pega doble.
- Tres luchadores por lado se eliminan correctamente y no quedan estados colgados.
- Las dificultades se distinguen sin cambiar atributos; la CPU no ralentiza la simulación.
- Touch concurrente se libera al cancelar; pausa/salida limpian ciclo y entradas.
- Victoria paga 7; derrota/empate 5; abandono 0; matchId repetido no paga.
- El perfil sobrevive recargas entre partidas; recargar en combate abandona sin premio. El sitio sirve con API apagada y recursos en caché offline.

Validación del repositorio

Al implementar, seguir AGENTS.md: pruebas Python de biblioteca estándar y node --check para JavaScript cambiado. Agregar pruebas con node:test para simulación y CPU; usar semillas para no depender del azar. QA manual: teclado, touch, orientación, tema claro/oscuro, pausa y rendimiento.

11. Registro de decisiones y referencias

Decisiones de producto estables

- Local, offline-capable, una persona contra CPU.
- Tres peleadores por equipo, uno activo, eliminación en cadena.
- Diez personajes, elegir tres sin repetir y ordenar.
- 40 s por duelo, hasta cinco duelos; timeout por vida y desempate de cinco segundos.
- 5 Cent por completar, +2 por victoria, 5 por empate, 0 por abandono.
- CPU de aprendizaje por refuerzo acotado; muy fácil, normal y difícil.
- Arena neo-retro de 16 bits, interfaz moderna inspirada en iOS.
- Motor separado de Flappy; sin dependencias externas.

Valores que se afinan con pruebas

Startup y recuperación; daño; alcance; reducción de bloqueo; chip; enfriamiento especial; latencia por dificultad; recompensa ML; escala y densidad de efectos. Cada valor tiene una sola fuente de verdad en configuración.

Archivos del proyecto revisados

- README.md: proyecto estático, almacenamiento de navegador, Cent, SQLite opcional y flujo de publicación.
- index.html y src/js/ui.js: lobby, pantalla de Flappy, router, perfil y resultado.
- src/js/game.js y src/css/game.css: Canvas, animación, pausa, carcasa vertical y controles.
- src/js/economy.js, api/economy.py, api/http_api.py: saldo, pago de Flappy y límite gameId.
- service-worker.js: lista de cache offline y versionado.

Referencias técnicas

Lecturas oficiales: tutoriales de TensorFlow.js y opciones de configuración. Se consideran referencia técnica; la primera versión evita esa dependencia.

Este plan guía la construcción del motor con figuras de prueba. Los sprites finales entran cuando las reglas, anclas, animaciones y hitboxes puedan probarse de forma precisa.

CenGames - 24 de octubre de 2026